دليل تعليمي للمبتدئين

الدليل الكامل لنشر موقع HTML / CSS / JavaScript باستخدام GitHub و Cloudflare Pages

من مجلد الملفات على جهازك حتى يصبح الموقع متاحًا على الإنترنت، مع نشر أي تعديل تلقائيًا بعد رفعه إلى GitHub.

Git · GitHub · Cloudflare Pages

هذا الدليل عام، ومكتوب لمن ينشر موقعًا ثابتًا من ملفات HTML و CSS و JavaScript فقط. لا يحتاج القارئ إلى خبرة سابقة في Git أو GitHub أو Cloudflare.

فهرس المحتويات

اقرأ الأقسام بالترتيب في المرة الأولى. بعد انتهاء الإعداد، ستكتفي غالبًا بالقسم 24 في كل مرة تعدّل فيها الموقع.

1. مقدمة 3
2. المتطلبات قبل البدء 4
3. تجهيز ملفات الموقع 5
4. إنشاء حساب GitHub 6
5. إنشاء Repository على GitHub 6
6. تثبيت Git على Windows 7
7. إعداد Git داخل مشروع الموقع 8
8. إضافة الملفات إلى Git 9
9. اختيار الفرع الرئيسي 10
10. ربط المشروع المحلي بـ GitHub 11
11. رفع المشروع لأول مرة إلى GitHub 11
12. التحقق من المشروع على GitHub 12
13. إنشاء حساب Cloudflare 13
14. ربط Cloudflare مع GitHub 13
15. إعداد Cloudflare Pages 14
16. إطلاق أول Deployment 15
17. اختبار الموقع 16
18. التحديثات المستقبلية 17
19. مثال كامل من البداية إلى النهاية 18
20. ربط Domain خاص 19
21. الأخطاء الشائعة وحلولها 20
22. الأمان 22
23. الفرق بين GitHub و Cloudflare 23
24. بعد الإعداد الأول، ماذا أفعل كل يوم؟ 23
25. قائمة تحقق نهائية 24
26. ملخص نهائي 25

1. مقدمة

هذا الدليل يشرح كيف تنشر موقعًا جاهزًا مبنيًا من ملفات HTML و CSS و JavaScript فقط. بعد الإعداد الأول، أي تعديل تحفظه ثم ترفعه بأمر واحد سيظهر على الموقع المنشور دون أن تعيد رفع الملفات يدويًا إلى الاستضافة.

ما هو Git؟

Git برنامج يعمل على جهازك. يتتبع تغييرات ملفات المشروع، ويحفظ كل مجموعة تغييرات كنقطة يمكن الرجوع إليها. هو الأداة التي تجهّز الملفات قبل إرسالها إلى الإنترنت.

ما هو GitHub؟

GitHub موقع يحفظ نسخة من مشروعك على الإنترنت داخل مكان يسمى Repository، أي مستودع. المستودع هو سجل المشروع: ملفاته وتاريخ تعديلاته. GitHub لا يستبدل الموقع المنشور للزوار بالضرورة؛ هو المكان الذي تعيش فيه النسخة المصدرية.

ما هو Cloudflare Pages؟

Cloudflare Pages خدمة تستضيف المواقع الثابتة وتنشرها. تقرأ الملفات من مستودع GitHub، ثم تجعلها متاحة على رابط عام. عندما يصلها تحديث جديد من GitHub، تنشره تلقائيًا.

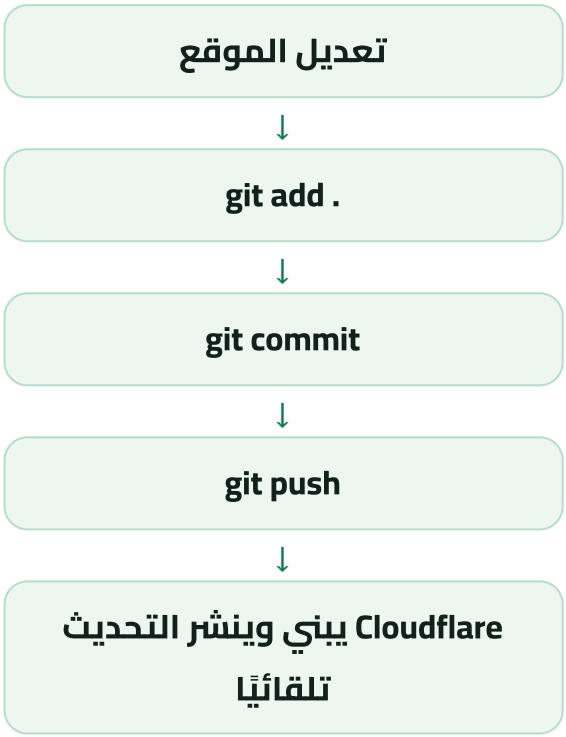
كيف ستعمل العملية كاملة؟



أنت تعدّل الملفات على جهازك. Git يسجّل التعديل. GitHub يستقبل النسخة. Cloudflare Pages يأخذ هذه النسخة وينشرها للزوار.

بعد الإعداد الأولي

لن تعيد خطوات الحسابات والربط في كل مرة. سير العمل اليومي يصبح:



معلومة مهمة

الإعداد الأول أطول لأنه يشمل الحسابات والتثبيت والربط. بعد ذلك، نشر التعديل عبارة عن ثلاثة أوامر داخل مجلد المشروع.

2. المتطلبات قبل البدء

جهّز هذه العناصر قبل أن تفتح أي أمر:

1. جهاز يعمل بنظام Windows أو macOS أو Linux.
2. موقع HTML و CSS و JavaScript جاهز، ويفضل أن يفتح بشكل صحيح على جهازك قبل النشر.
3. حساب GitHub.
4. برنامج Git مثبّت على الجهاز.
5. حساب Cloudflare.
6. اتصال بالإنترنت.

ملاحظة

يمكن إنشاء حساب GitHub وحساب Cloudflare مجانًا للبدء. الخطة المجانية كافية لنشر موقع ثابت بسيط وربطه بالمستودع.

خطوات تثبيت Git المفضّلة في هذا الدليل مكتوبة لـ Windows، لأن أوامر المجلد فيها تستخدم مسارًا مثل `C:\Users\...`. أوامر Git نفسها بعد ذلك هي نفسها على الأنظمة الثلاثة. إذا كنت على macOS أو Linux، بدّل أمر `cd` فقط إلى مسار مجلدك.

3. تجهيز ملفات الموقع

قبل Git، تأكد أن ملفات الموقع مرتبة. الشكل المفضل لموقع بسيط هو أن تكون الصفحة الرئيسية في المجلد الأعلى للمشروع:

```
my-website/
│
├── index.html
├── style.css
├── script.js
├── images/
│   ├── logo.png
│   └── ...
└── ...
```

الأسماء هنا أمثلة. قد يكون ملف التنسيق داخل مجلد `css` وملف JavaScript داخل مجلد `js`. المهم هو مكان `index.html`.

لماذا index.html في المجلد الرئيسي؟

عندما يفتح الزائر رابط الموقع، يبحث الخادم عن صفحة اسمها `index.html` ويعرضها تلقائيًا. إذا كان هذا الملف في المجلد الذي تنشره Cloudflare، تظهر الصفحة الرئيسية دون أن يكتب الزائر اسم الملف.

Cloudflare لا يخترع مكان الملفات. ينشر المجلد الذي تحدده أنت في إعدادات المشروع. إذا كان `index.html` في جذر المستودع، اترك Root directory فارغًا. إذا كان داخل مجلد فرعي، أخبر Cloudflare باسم ذلك المجلد.

إذا كان الموقع داخل مجلد فرعي

```
my-website/
└── website/
    ├── index.html
    ├── style.css
    └── script.js
```

في هذه الحالة اضبط Root directory في Cloudflare على:

```
website
```

> **تحذير**
>
> إذا تركت Root directory فارغًا بينما `index.html` موجود داخل `website` فقط، قد ينجح النشر وتظهر صفحة فارغة أو غير صحيحة، لأن Cloudflare ينظر إلى المجلد الخطأ.

4. إنشاء حساب GitHub

إذا كان لديك حساب GitHub وتعمل عليه، تجاوز هذا القسم وانتقل إلى إنشاء المستودع.

1. افتح المتصفح واذهب إلى `https://github.com`
2. اضغط **Sign up**.
3. أدخل بريدًا إلكترونيًا تستطيع فتحه.
4. أنشئ كلمة مرور للحساب واحفظها في مكان آمن عندك. لا تكتبها داخل ملفات الموقع.
5. اختر Username. هذا الاسم سيظهر في رابط مشاريعك، مثل `github.com/USERNAME`.
6. أكمل التحقق الذي يطلبه الموقع، ثم أكّد البريد الإلكتروني من الرسالة التي تصل إليك.
7. ارجع إلى GitHub وسجّل الدخول.

> **ملاحظة**
>
> Username يختلف عن اسمك الظاهر. يمكن تغيير الاسم الظاهر لاحقًا، أما اسم المستخدم فهو جزء من روابط المستودعات، فاختره بوضوح.

5. إنشاء Repository على GitHub

المستودع هو العلبة التي سترفع إليها ملفات الموقع. أنشئه فارغًا تقريبًا، ثم ارفع ملفات جهازك إليه.

1. سجّل الدخول إلى GitHub.
2. من الصفحة الرئيسية أو من قائمة **+** اختر **New repository**.
3. اكتب اسم المشروع في **Repository name**.
4. اختر **Public** إذا كان يجوز لأي شخص رؤية الملفات، أو **Private** إذا أردت إبقاء المستودع خاصًا. Cloudflare يستطيع النشر من الاثنين بعد منحه الصلاحية.
5. لا تفعّل إضافة README أو `.gitignore` أو license إذا كان المشروع موجودًا أصلًا على جهازك.
6. اضغط **Create repository**.

مثال على الاسم:

```
my-website
```

بعد الإنشاء ستظهر صفحة فيها أوامر جاهزة ورابط ينتهي غالبًا بـ `.git`. انسخ الرابط الذي يبدأ بـ `https://github.com/` واحفظه. ستحتاجه في قسم الربط. شكله:

```
https://github.com/USERNAME/my-website.git
```

معلومة مهمة

استبدل `USERNAME` باسم مستخدمك الحقيقي، و`my-website` باسم المستودع الحقيقي. الرابط في هذا الدليل مثال فقط.

تحذير

إذا أنشأت README من موقع GitHub ثم حاولت أول رفع من جهاز فيه ملفات أخرى، قد يرفض Git الدمج لأن الطرفين بدآ بتاريخين مختلفين. الحل الآمن مشروح في قسم الأخطاء. الأسهل لمشروع موجود على الجهاز: أنشئ المستودع بدون README.

6. تثبيت Git على Windows

Git ليس جزءًا ظاهرًا من المتصفح. تثبّته مرة واحدة على الجهاز.

هل Git مثبّت أصلًا؟

افتح PowerShell أو CMD. في Windows ابحث عن PowerShell من قائمة ابدأ وافتحه، ثم اكتب:

```
git --version
```

هذا الأمر يسأل الجهاز عن إصدار Git. إذا ظهر سطر مثل:

```
git version 2.x.x
```

فـ Git مثبّت، ويمكنك الانتقال إلى القسم التالي. الرقم الدقيق يختلف حسب نسخة التثبيت، وأي إصدار حديث من السلسلة 2 مناسب.

إذا ظهر أن الأمر غير معروف

إذا ظهرت رسالة مثل `git is not recognized`، فـ Git غير مثبّت أو أن نافذة الطرفية فُتحت قبل التثبيت.

1. افتح الموقع الرسمي: `https://git-scm.com/`
2. نزّل **Git for Windows** من صفحة التنزيل الرسمية، لا من مصدر غير معروف.
3. شغّل المثبّت. الإعدادات الافتراضية مناسبة للمبتدئ. اترك خيار Git Credential Manager مفعّلًا إن ظهر، لأنه يفتح نافذة تسجيل الدخول لاحقًا بدل كتابة كلمة المرور داخل الأمر.
4. بعد انتهاء التثبيت، أغلق PowerShell تمامًا ثم افتح نافذة جديدة.
5. أعد تنفيذ:

```
git --version
```

حل المشكلة

إغلاق النافذة القديمة ضروري. النافذة التي كانت مفتوحة قبل التثبيت قد لا ترى Git حتى بعد نجاح التثبيت.

ملاحظة

على macOS، نفّذ `git --version` في Terminal. إذا طُلب منك تثبيت أدوات سطر الأوامر فوافق، أو نزّل Git من الموقع نفسه. على Ubuntu يمكن استخدام `sudo apt update` ثم `sudo apt install git`. بعد ذلك تبقى أوامر هذا الدليل كما هي.

7. إعداد Git داخل مشروع الموقع

يجب أن تكون نافذة PowerShell واقفة داخل مجلد الموقع نفسه، لا داخل مجلد سطح المكتب العام ولا داخل مجلد المستخدم.

مثال إذا كان المشروع هنا:

```
C:\Users\User\Desktop\my-website
```

اكتب، مع الإبقاء على علامات الاقتباس لأن المسار قد يحتوي مسافات:

```
cd "C:\Users\User\Desktop\my-website"
```

الأمر `cd` ينقل الطرفية إلى المجلد. غيّر المسار ليطابق مكان مشروعك. للتأكد أنك في المكان الصحيح، نفّذ `dir` على Windows. يجب أن ترى `index.html` ضمن النتائج.

بدء المستودع المحلي

```
git init
```

هذا الأمر ينشئ مستودع Git داخل المجلد الحالي. يضيف مجلدًا مخفيًا اسمه `.git` يحفظ فيه السجل. لا يحذف ملفات الموقع ولا يرفعها بعد إلى الإنترنت.

رؤية الحالة

```
git status
```

هذا الأمر يعرض حالة المشروع الآن: أي الملفات جديدة، وأيها معدّلة، وأيها جاهزة للحفظ. بعد `git init` مباشرة ستظهر ملفات الموقع غالبًا كملفات غير متتبَّعة، أي Untracked، لأن Git رآها ولم تُضَف بعد إلى أي حفظ.

> **ملاحظة**
>
> إذا قال Git إنك لست داخل مستودع، فأنت في المجلد الخطأ أو لم يُنفَّذ `git init` هنا. راجع قسم الأخطاء.

8. إضافة الملفات إلى Git

Git لا يحفظ الملفات في سجل المشروع بمجرد وجودها في المجلد. أولًا تضعها في منطقة تحضير اسمها Staging، ثم تصنع Commit.

```
git add .
```

النقطة تعني «هذا المجلد وما بداخله». الأمر يجهّز الملفات الحالية للحفظ. لا يرسلها إلى GitHub بعد.

```
git status
```

بعد `git add .` يفترض أن ترى الملفات تحت عنوان يشبه Changes to be committed. هذا يعني أنها في منطقة التحضير. إذا بقي ملف مهم خارج هذه القائمة، فتأكد أنك داخل المجلد الصحيح ثم أعد `git add .`.

تعريف الاسم الذي سيظهر في السجل

قبل أول Commit، يحتاج Git إلى اسم وبريد يضعهما على نقطة الحفظ. هذان ليسا كلمة مرور GitHub، ولا يُرفعان كسر. نفّذهما مرة واحدة على هذا الجهاز إذا لم تكونا مضبوطين:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

استبدل الاسم والبريد ببياناتك. البريد يفضَّل أن يكون البريد نفسه المستخدم في GitHub حتى تُنسب إليك المساهمات بوضوح. `--global` يعني أن الإعداد لكل مشاريع هذا الجهاز، فلا تحتاج إلى إعادته في كل مجلد.

صنع أول نقطة حفظ

```
git commit -m "Initial commit"
```

الـ Commit لقطة محفوظة للملفات التي في منطقة التحضير، مع رسالة تصفها. `-m` يضع الرسالة مباشرة. العبارة Initial commit تعني أن هذه أول لقطة للمشروع. بعد نجاح الأمر لن تكون الملفات على GitHub بعد؛ هي محفوظة في Git على جهازك فقط.

> **تحذير**
>
> لا تضع كلمات مرور أو مفاتيح سرية داخل الملفات ثم تعمل لها Commit. ما يدخل إلى Git ثم إلى GitHub يبقى في السجل حتى لو حذفت الملف لاحقًا من آخر نسخة.

9. اختيار الفرع الرئيسي

```
git branch -M main
```

الفرع Branch هو خط العمل. `main` هو الاسم الذي سنستخدمه للخط الرئيسي. الأمر `-M` يسمّي الفرع الحالي `main`، أو يعيد تسميته إذا كان اسمه القديم `master`. Cloudflare سينشر من هذا الفرع، لذلك يجب أن يتطابق الاسم هنا مع الفرع الذي تختاره لاحقًا في Cloudflare.

ملاحظة

في نسخ Git الحديثة قد يكون اسم الفرع `main` منذ `git init`. تنفيذ الأمر مع ذلك آمن، ويثبّت الاسم الذي سنعتمد عليه في بقية الدليل.

10. ربط المشروع المحلي بـ GitHub

حتى الآن Git يعمل على جهازك فقط. الربط يخبره بعنوان المستودع على GitHub. انسخ رابط المستودع من صفحة GitHub، ثم استبدل المثال برابطك:

```
git remote add origin https://github.com/USERNAME/my-website.git
```

`remote` يعني وجهة بعيدة. `add` يضيفها. `origin` هو الاسم المحلي المعتاد لهذه الوجهة، مثل لقب للرابط. الرابط نفسه هو عنوان المستودع.

للتحقق:

```
git remote -v
```

الأمر `-v` يعرض الوجهات بالتفصيل. النتيجة الصحيحة تشبه:

```
origin  https://github.com/USERNAME/my-website.git (fetch)
origin  https://github.com/USERNAME/my-website.git (push)
```

`fetch` هو عنوان السحب، و`push` هو عنوان الإرسال. يجب أن يكون الرابطان هما رابط مستودعك، واسم المستخدم واسم المشروع صحيحين حرفًا بحرف.

حل المشكلة

إذا ظهر `remote origin already exists` فاللقب origin مستخدم من قبل. اعرض الرابط بـ `git remote -v`. إذا كان خاطئًا، صححه بهذا الأمر بدل إضافة origin مرة ثانية:

```
git remote set-url origin https://github.com/USERNAME/my-website.git
```

11. رفع المشروع لأول مرة إلى GitHub

```
git push -u origin main
```

هذا الأمر يرسل فرع `main` من جهازك إلى الوجهة المسماة `origin`، أي إلى GitHub. `-u` يحفظ علاقة بين فرعك المحلي وفرع `main` على GitHub، فيعرف Git في المرات التالية أين يرسل التحديث.

في المرة الأولى قد تتوقف النافذة وتطلب تسجيل الدخول. هذا طبيعي.

معلومة مهمة

GitHub لم يعد يقبل كلمة مرور الحساب العادية داخل أوامر Git عبر HTTPS. غالبًا ستظهر نافذة Git Credential Manager أو صفحة تسجيل دخول في المتصفح. سجّل الدخول منها بحسابك. لا تكتب كلمة المرور داخل سطر الأمر، ولا تضع أي Token داخل ملفات المشروع أو داخل هذا الدليل.

إذا ظهر حقل Password ولم تفتح نافذة متصفح، فالمطلوب عادة رمز Personal Access Token تنشئه من إعدادات GitHub لديك، لا كلمة مرور تسجيل الدخول إلى الموقع. أنشئ الرمز من حسابك، والصقه في حقل كلمة المرور عندما يُطلب، ثم لا تحفظه داخل HTML أو JavaScript أو أي ملف سيُرفع.

عند النجاح تظهر أسطر تدل على أن الفرع أُرسل، مثل إشارة إلى `main` و`origin/main`. إذا توقف الأمر برسالة rejected، لا تعِد المحاولة بقوة. انتقل إلى قسم الأخطاء.

12. التحقق من المشروع على GitHub

افتح صفحة المستودع في المتصفح:

```
https://github.com/USERNAME/my-website
```

حدّث الصفحة. يجب أن ترى ملفات الموقع، مثل:

- `index.html`
- `style.css` أو مجلد التنسيق الذي تستخدمه
- `script.js` أو مجلد JavaScript الذي تستخدمه
- `images/` إذا كانت الصور داخل هذا المجلد

وهذه بقية ملفات الموقع التي عملت لها Commit. إن نقص ملف، ارجع إلى الجهاز ونفّذ `git status`. إذا كان الملف ما زال غير متتبَّع، فأضفه بـ `git add .` ثم Commit ثم:

```
git push origin main
```

ملاحظة

ظهور الملفات على GitHub يعني أن النسخة المصدرية وصلت. الزوار لن يفتحوا الموقع من رابط GitHub هذا كموقع جاهز إلا إذا فعّلت لاحقًا أداة أخرى. رابط الزوار سيأتي من Cloudflare Pages.

13. إنشاء حساب Cloudflare

1. افتح `https://www.cloudflare.com/` أو `https://dash.cloudflare.com/`
2. اضغط **Sign up**.
3. أنشئ الحساب بالبريد وكلمة المرور، أو سجّل الدخول إذا كان الحساب موجودًا.
4. أكّد البريد الإلكتروني إذا وصلت رسالة تطلب ذلك.

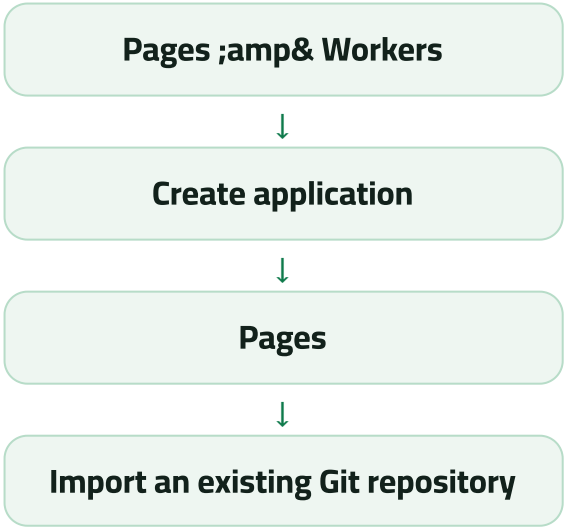
بعد الدخول تصل إلى لوحة Cloudflare. منها ستستخدم Cloudflare Pages لاستضافة الموقع الثابت: صفحات وملفات جاهزة تُعرض كما هي، بلا برنامج سيرفر يبنيها في كل طلب.

ملاحظة

أسماء بعض القوائم في لوحة Cloudflare تتغير بين فترة وأخرى. إذا لم تجد العبارة حرفيًا، ابحث داخل اللوحة عن Pages أو عن Create application. المعنى المطلوب هو إنشاء مشروع Pages من مستودع Git موجود.

14. ربط GitHub مع Cloudflare

المسار المعتاد داخل اللوحة:



1. من لوحة Cloudflare افتح **Pages ;amp& Workers**.
2. اختر **Create application** أو **Create**.

3. اختر تبويب **Pages**، لا تبويب Worker إذا كان الهدف موقع ملفات ثابتة.
4. اختر **Connect to Git** أو **Import an existing Git repository**.
5. اختر **GitHub**.
6. وافق على صلاحية الوصول التي يطلبها Cloudflare. هذه الصلاحية تسمح له بقراءة المستودع الذي ستختاره حتى يستطيع النشر.
7. إذا طُلب منك تحديد الحساب أو Organization، اختر الحساب الذي يملك المستودع.
8. اختر Repository الخاص بالموقع، مثل `my-website`.
9. تابع إلى شاشة إعداد البناء. لا تضغط حفظًا نهائيًا قبل مراجعة القسم التالي.

> **تحذير**
> إذا لم يظهر المستودع في القائمة، فغالبًا صلاحية GitHub لا تشمل هذا المستودع. ارجع إلى شاشة الربط واختر ضبط صلاحيات GitHub، ثم اسمح للمستودع المطلوب، سواء كان Public أو Private.

15. إعداد Cloudflare Pages لموقع HTML/CSS/JS

هذا الموقع Static Website. ملفاته جاهزة للعرض. لا يوجد إطار عمل يبنيها، ولا خطوة تجميع قبل النشر.

في شاشة Set up builds and deployments استخدم القيم التالية:

الإعداد	القيمة لهذا المشروع
Project name	اسم واضح، مثل my-website. سيؤثر في رابط pages.dev
Production branch	main
Framework preset	None
Build command	اتركه فارغًا
Build output directory	الشرطة المائلة أو النقطة، أي جذر المشروع
Root directory	فارغ إذا كان index.html في جذر المستودع
Environment variables	لا تضف شيئًا لموقع HTML/CSS/JS بسيط

Framework preset بقيمة None يعني أن Cloudflare لن يفترض أن المشروع React أو أي أداة أخرى، ولن يملأ أمر بناء لا تحتاجه.

Build command هو أمر يُشغَّل قبل النشر لتصنيع الملفات. موقعك لا يحتاج تصنيعًا، لذلك يُترك الحقل فارغًا. إذا رفضت الواجهة تركه فارغًا، اكتب الأمر التالي فقط، وهو أمر ينجح فورًا ولا يبني شيئًا:

```
exit 0
```

Build output directory هو المجلد الذي ستُنشر محتوياته. عندما تكون القيمة الشرطة المائلة أو النقطة، ينشر Cloudflare جذر المشروع، حيث يوجد `index.html`. إذا اختلف نص الحقل في واجهتك، اختر القيمة التي تعني المجلد الحالي نفسه.

Root directory هو المجلد الذي يبدأ منه Cloudflare داخل المستودع. اتركه فارغًا عندما يكون `index.html` في أعلى المستودع. إذا كانت الملفات داخل مجلد اسمه `website`، اكتب:

```
website
```

معلومة مهمة

هذه الإعدادات خاصة بموقع HTML و CSS و JavaScript ثابت. لا تستخدمها كما تُستخدم لمشاريع أخرى. في هذا المشروع لا نضع أمر بناء خاصًا بإطار عمل، ولا نحدد مجلد ناتج بناء مثل مجلد إنتاج لأداة خارجية.

تحذير

لا تملأ Build command بأوامر لا تخص موقعك، ولا تضبط مجلد الإخراج على مجلد غير موجود. أكثر سبب لفشل أول نشر هو مجلد إخراج أو جذر خاطئ، لا عطل في الملفات نفسها.

16. إطلاق أول Deployment

بعد مراجعة الإعدادات اضغط **Save and Deploy**. قد يختلف نص الزر قليلًا، مثل Deploy site. المعنى واحد: ابدأ أول نشر.

ماذا يحدث الآن؟ Cloudflare يسحب الملفات من فرع `main` على GitHub. إذا كان أمر البناء فارغًا أو `exit 0`، لا يصنع ملفات جديدة. ثم يرفع محتويات مجلد الإخراج إلى شبكته وينشرها.

انتظر حتى تصبح حالة النشر Success. بعدها يظهر رابط عام يشبه:

```
https://my-website.pages.dev
```

الاسم قبل `pages.dev` هو اسم مشروع Cloudflare، وقد يختلف عن اسم المستودع إذا غيّرته في خانة Project name. هذا الرابط هو عنوان الموقع للزوار في الخطة المجانية، قبل ربط أي نطاق خاص.

> **ملاحظة**
>
> أول نشر قد يأخذ دقيقة أو أكثر. إذا بقيت الحالة Building، لا تعِد إنشاء المشروع. افتح تفاصيل هذا الـ Deployment وانتظر أو اقرأ السجل إن فشل.

17. اختبار الموقع

افتح رابط `pages.dev` في المتصفح، وجرّبه أيضًا من هاتف أو من نافذة ضيقة.

- هل ظهرت صفحة `index.html` وليس صفحة خطأ؟
- هل الشكل مطابق للتنسيق، أي هل تحمّل CSS؟
- هل تعمل الأزرار والقوائم التي يعتمد عليها JavaScript؟
- هل تظهر الصور؟
- هل تعمل الروابط الداخلية بين الصفحات؟
- هل الموقع سليم على الهاتف وعلى الكمبيوتر؟

إذا ظهرت الصفحة بلا تنسيق

غالبًا مسار ملف CSS داخل HTML لا يطابق مكان الملف. مثال قد يسبب مشكلة في بعض أنواع النشر، خصوصًا إذا نُشر الموقع لاحقًا تحت مسار فرعي:

```
/style.css
```

الشرطة في البداية تعني «من جذر الموقع». هذا يعمل على Cloudflare عندما يُنشر الموقع على جذر النطاق وملف CSS موجود هناك. للموقع البسيط، المسار النسبي أوضح وأقل مفاجأة إذا كان الملف بجانب `index.html`:

```
style.css
```

وإذا كان الملف داخل مجلد:

```
css/styles.css
```

يجب أن يطابق هذا المسار الاسم الحقيقي للمجلد والملف، بما في ذلك الأحرف الكبيرة والصغيرة.

الصور

السبب نفسه يمنع ظهور صورة: اسم مختلف، مجلد مختلف، أو حرف كبير في الجهاز وحرف صغير في الكود. Cloudflare يعمل على نظام يفرّق بين `Logo.png` و `logo.png`.

حل المشكلة

افتح أدوات المطوّر في المتصفح وتبويب Network. إذا ظهر ملف CSS أو صورة باللون الأحمر، فالمسار أو الاسم غير مطابق. صحح المسار في الملف، ثم اتبع قسم التحديثات لرفع التصحيح.

18. التحديثات المستقبلية

هذا هو الجزء الذي ستستخدمه بعد اليوم الأول. لا تُعد رفع الملفات يدويًا إلى Cloudflare، ولا تعِد إنشاء المشروع.

1. عدّل الملفات على جهازك واحفظها.
2. افتح PowerShell داخل مجلد المشروع.
3. انظر ماذا تغيّر:

```
git status
```

ستظهر الملفات التي عدّلتها. ثم جهّزها:

```
git add .
```

ثم احفظ لقطة برسالة تصف التعديل. غيّر النص بين علامتي الاقتباس حسب ما فعلت:

```
git commit -m "Update website"
```

ثم أرسل اللقطة إلى GitHub:

```
git push origin main
```

Cloudflare Pages يراقب فرع `main`. عندما يصل Push جديد، يبدأ Deployment جديد وحده. بعد نجاحه يتحدث الموقع على الرابط نفسه.

معلومة مهمة

لن ترى التعديل على الموقع في اللحظة نفسها دائمًا. انتظر انتهاء النشر من تبويب Deployments. إذا بقي الموقع القديم بعد حالة Success، حدّث الصفحة تحديثًا كاملًا في المتصفح.

ملاحظة

إذا نفّذت `git commit` ونسيت `git push`، فالتعديل موجود على جهازك فقط ولن يعرفه Cloudflare.

19. مثال كامل من البداية إلى النهاية

المثال التالي افتراضي. المجلد:

```
C:\Users\User\Desktop\my-website
```

وفيه `index.html` مباشرة. اسم المستخدم على GitHub هو `USERNAME` واسم المستودع `my-website`. بدّلهما قبل التنفيذ.

على الجهاز

```
cd "C:\Users\User\Desktop\my-website"
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/USERNAME/my-website.git
git push -u origin main
```

قبل `git commit`، إذا طُلب منك اسم المستخدم لأول مرة، نفّذ إعداد `user.name` و`user.email` المذكور في القسم 8. وأثناء `git push` سجّل الدخول من النافذة التي تظهر.

على Cloudflare

- ☐ Workers & Pages ثم Create ثم Pages
- ☐ Import an existing Git repository وربط GitHub
- ☐ اختيار مستودع my-website
- ☐ Production branch = main
- ☐ Framework preset = None
- ☐ Build command فارغ
- ☐ Build output directory = / أو .
- ☐ Root directory فارغ
- ☐ Save and Deploy

- [ ] فتح رابط pages.dev واختبار الصفحة والتنسيق والصور

عند أي تحديث لاحق

```
git add .
git commit -m "Update website"
git push origin main
```

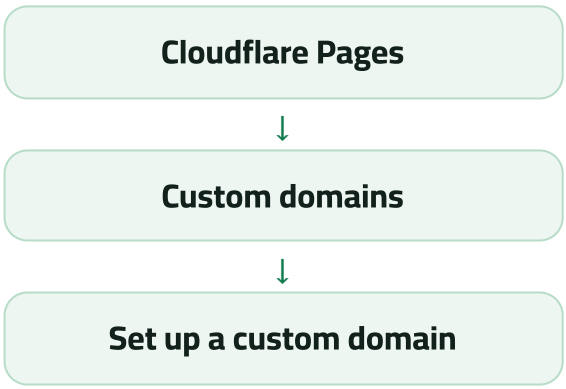
ثم انتظر Deployment الجديد في Cloudflare وافتح الموقع للتأكد.

20. ربط Domain خاص

الرابط `my-website.pages.dev` مجاني من Cloudflare ويكفي لعمل الموقع ومشاركته. لاحقًا يمكن عرض الموقع على نطاق تمتلكه، مثل:

```
www.example.com
```

المسار داخل المشروع المنشور:



1. افتح مشروع Pages الخاص بالموقع.
2. ادخل إلى **Custom domains**.
3. اختر **Set up a custom domain**.
4. اكتب النطاق، مثل `www.example.com`، ثم تابع التعليمات التي تعرضها Cloudflare.

إعداد DNS يعتمد على المكان الذي اشتريت منه النطاق والمكان الذي تديره منه. إذا كان النطاق أصلًا على Cloudflare، غالبًا تُضاف السجلات لك أو بخطوات قصيرة. إذا كان عند شركة أخرى، ستعرض Cloudflare القيم التي يجب إضافتها هناك، عادة سجل CNAME يشير إلى موقع Pages.

ملاحظة

لا تحذف سجلات البريد في نطاقك أثناء ربط الموقع إذا كنت لا تعرف وظيفتها. انسخ القيم التي يطلبها Cloudflare للنطاق أو لـ www فقط، واترك سجلات البريد كما هي إلى أن تتأكد.

بعد اكتمال الربط قد يحتاج ظهور النطاق الجديد إلى بعض الوقت. رابط `pages.dev` يبقى يعمل أيضًا.

21. الأخطاء الشائعة وحلولها

1. git is not recognized

PowerShell لا يجد برنامج Git.

حل المشكلة

ثبّت Git for Windows من `https://git-scm.com/` ثم أغلق PowerShell وافتحه من جديد، وجرّب `git --version`.

2. fatal: not a git repository

أنت خارج مجلد المشروع، أو لم يبدأ Git داخل هذا المجلد.

حل المشكلة

انتقل إلى مجلد الموقع وتأكد أن `dir` يُظهر `index.html`، ثم نفّذ:

```
git init
```

3. remote origin already exists

اسم origin محجوز برابط قديم.

حل المشكلة

اعرض الرابط الحالي:

```
git remote -v
```

إذا كان خاطئًا فصححه، ولا تضف origin مرة أخرى:

```
git remote set-url origin https://github.com/USERNAME/my-website.git
```

4. rejected push

يرفض GitHub الاستلام غالبًا لأن على المستودع البعيد Commit ليس عندك، مثل README أُنشئ من الموقع. الرسالة قد تذكر fetch first أو non-fast-forward.

حل المشكلة

اسحب ما على GitHub أولًا ثم أعد الإرسال:

```
git pull origin main --rebase
git push origin main
```

إذا قال Git إن التاريخين غير مرتبطين، لأن المستودع على الموقع بدأ بملف وأنت بدأت بـ `git init` منفصل:

```
git pull origin main --rebase --allow-unrelated-histories
git push origin main
```

تحذير

لا تستخدم `git push --force` كحل أول. هذا الأمر قد يمسح تاريخ المستودع على GitHub. الخطوات أعلاه تدمج الطرفين بدل مسح أحدهما.

5. الموقع يظهر بدون CSS

HTML وصل، وملف التنسيق لم يُعثر عليه.

حل المشكلة

طابق مسار CSS في `index.html` مع مكان الملف الحقيقي. للموقع البسيط فضّل مسارًا نسبيًا مثل `style.css` أو `css/styles.css`، وتجنب مسارًا يبدأ من جذر الموقع بشرطة مائلة إذا كان النشر أو التجربة يتمان من مجلد فرعي. تأكد أيضًا أن ملف CSS نفسه رُفع إلى GitHub.

6. الصور لا تظهر

حل المشكلة

راجع اسم الملف، واسم المجلد، وحالة الأحرف. `images/logo.png` لن يفتح `images/Logo.PNG` على Cloudflare. تأكد أن مجلد الصور دخل إلى Commit وظهر على GitHub.

7. Cloudflare Deployment يفشل

افتح النشر الفاشل واقرأ الأسطر الأخيرة في السجل. ثم راجع:

- هل Repository هو مستودع هذا الموقع؟
- هل Production branch هو `main`، وهو نفسه الفرع الذي ترفعه؟
- هل Root directory فارغ أو يساوي المجلد الذي فيه `index.html`؟
- هل Build command فارغ أو `exit 0` فقط، بلا أوامر لا تخص المشروع؟
- هل Output directory يعني جذر المشروع، بشرطة مائلة أو بنقطة، وليس مجلدًا غير موجود؟

8. Cloudflare يعرض صفحة غير صحيحة

حل المشكلة

تأكد من وجود `index.html` في المكان الذي ينشره Cloudflare. إذا كان الملف داخل مجلد فرعي، اضبط Root directory على اسم ذلك المجلد ثم أعد النشر.

9. التعديل على GitHub لا يظهر فورًا على Cloudflare

حل المشكلة

النشر يحتاج وقتًا. افتح المشروع في Cloudflare ثم Deployments وتأكد أن هناك نشرًا جديدًا بحالة Success بعد وقت الـ Push. إذا لم يبدأ نشر جديد، فتحقق أن الدفع ذهب إلى فرع `main` وأن المشروع مربوط بهذا المستودع وهذا الفرع. بعد النجاح حدّث المتصفح تحديثًا كاملًا.

10. فشل تسجيل الدخول أثناء git push

حل المشكلة

لا تعِد كتابة كلمة مرور حساب GitHub داخل الأمر. أعد `git push -u origin main` وسجّل الدخول من نافذة Git Credential Manager أو المتصفح. تأكد من رابط `git remote -v`. إذا طُلب حقل كلمة مرور، استخدم Personal Access Token من إعدادات GitHub، ولا تحفظ هذا الرمز داخل ملفات الموقع.

22. الأمان

كل ما تضعه في HTML أو CSS أو JavaScript ويُنشر مع الموقع يصل إلى متصفح الزائر. يستطيع أي شخص فتح أدوات المطوّر وقراءة هذا الكود. لذلك لا تضع أسرارًا فيه.

مثال سيئ، لا تكتبه في مشروعك:

```
const apiKey = "SECRET_KEY";
```

حتى لو كان المفتاح في سطر واحد وبدا مخفيًا، فهو ليس مخفيًا بعد النشر.

معلومة مهمة

Environment variables في إعدادات Cloudflare لا تجعل القيمة سرية إذا وُضعت لاحقًا داخل JavaScript الذي يصل إلى المتصفح. المتغير قد يفيد أثناء البناء، لكن الناتج النهائي الذي يحمّله الزائر يمكن قراءته. أي مفتاح سري يجب ألا يُشحن إلى الواجهة.

إذا احتاج الموقع إلى مفتاح API سري، فالمكان الصحيح له هو خادم أو واجهة خلفية تستدعي الخدمة نيابة عن المتصفح، ثم يعيد المتصفح النتيجة فقط. موقع HTML/CSS/JS البسيط بلا خادم لا يخفي هذا النوع من المفاتيح.

تحذير

لا ترفع ملفًا فيه كلمات مرور أو رموز وصول أو مفاتيح خاصة. إذا رُفع بالخطأ، فحذف السطر في Commit جديد لا يمسحه من التاريخ القديم. عطّل المفتاح من الخدمة التي أصدرته، ثم نظّف المستودع بطريقة تناسب حالتك.

23. الفرق بين GitHub و Cloudflare

الأداة	وظيفتها في هذا الدليل
Git	البرنامج على جهازك. يسجّل التعديلات ويرسلها إلى GitHub.
GitHub	يحفظ الكود وتاريخ الإصدارات داخل Repository.
Cloudflare Pages	يستضيف الموقع وينشره للزوار، ويعيد النشر عندما يصل تحديث إلى GitHub.

العلاقة خط واحد: تعدّل الملفات، Git يحفظ اللقطة، GitHub يستقبلها، Cloudflare Pages يقرأها من GitHub ويعرضها. GitHub هنا مصدر الملفات. Cloudflare هو عنوان الزيارة. Git هو الجسر بين مجلدك وGitHub.

لذلك لا يكفي أن تنجح أوامر Git حتى يفتح الزائر الموقع، ولا يكفي إنشاء حساب Cloudflare دون ربطه بالمستودع الصحيح. السلسلة تكتمل عندما ينجح أول Deployment على رابط `pages.dev`.

24. بعد الإعداد الأول، ماذا أفعل كل يوم؟

عندما تكون الحسابات والربط وأول نشر قد اكتملت، لا تعد الأقسام السابقة. في كل تعديل:

1. أعدّل الملفات.
2. احفظ الملفات في المحرر.
3. افتح PowerShell داخل مجلد المشروع.
4. نفّذ:

```
git add .
```

5. ثم:

```
git commit -m "Describe changes"
```

استبدل `Describe changes` بجملة قصيرة عمّا تغيّر.

6. ثم:

```
git push origin main
```

7. انتظر Cloudflare Deployment حتى يصبح Success.
8. الموقع يتحدث تلقائيًا على الرابط نفسه.

> **ملاحظة**
> يمكن البدء بـ `git status` قبل `git add .` لترى أسماء الملفات التي ستُحفظ. هذا لا يغيّر شيئًا، ويعطيك فرصة للتراجع قبل الـ Commit إذا ظهر ملف لا تريد رفعه.

25. قائمة تحقق نهائية

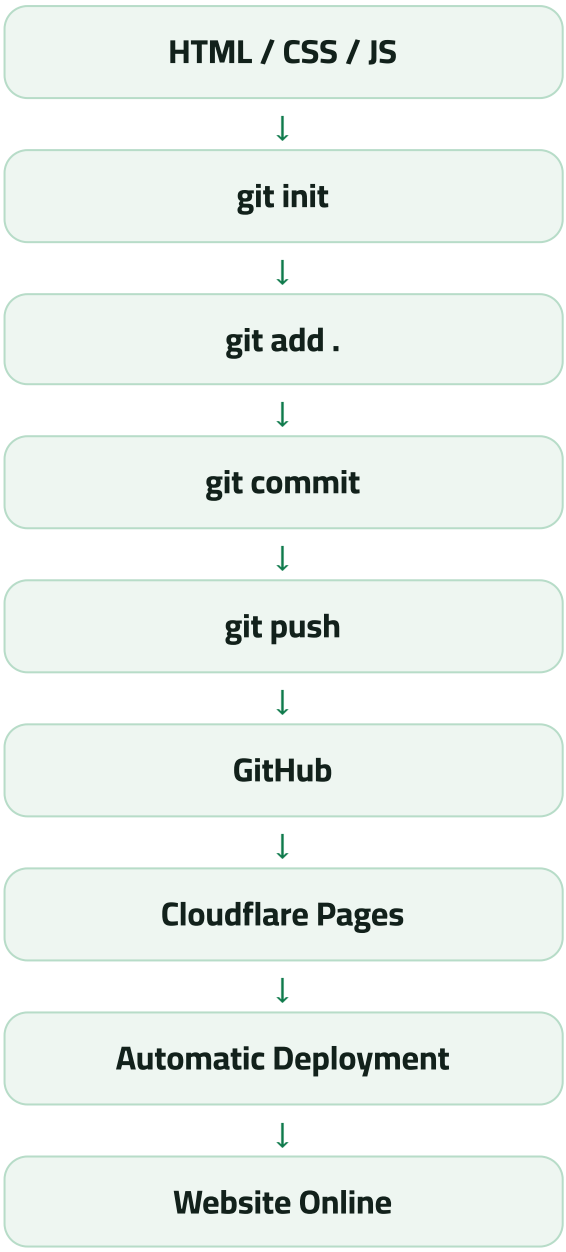
يمكن طباعة هذه الصفحة وتعليم كل بند بعد إنجازه.

- ☐ الموقع يحتوي على index.html
- ☐ Git مثبّت
- ☐ حساب GitHub جاهز
- ☐ تم إنشاء Repository
- ☐ تم تنفيذ git init
- ☐ تمت إضافة الملفات

- ☐ تم إنشاء Initial commit
- ☐ main = branch
- ☐ المستودع مربوط بـ origin
- ☐ أول Push نجح
- ☐ حساب Cloudflare جاهز
- ☐ GitHub مربوط بـ Cloudflare
- ☐ تم اختيار Repository الصحيح
- ☐ None = Framework preset
- ☐ Build command فارغ
- ☐ Output directory مضبوط
- ☐ Deployment نجح
- ☐ رابط pages.dev يعمل
- ☐ تم ربط Custom domain لاحقًا إن وُجد

26. ملخص نهائي

العملية كلها خط واحد. الإعداد الأول يمشي من أعلى المخطط إلى أسفله. بعد ذلك تعيد فقط الجزء من تعديل الملفات حتى git push، ويتولى Cloudflare الباقي.



إذا توقفت في خطوة، ارجع إلى قسم الأخطاء وابحث عن نص الرسالة كما ظهر في PowerShell أو في سجل Cloudflare. انسخ الرسالة كما هي؛ هي أدق من تخمين السبب.

ملاحظة

احتفظ برابط المستودع ورابط pages.dev في مكان تعرفه. الرابط الأول لإدارة الملفات، والرابط الثاني هو الموقع الذي يفتحه الزائر.